ANALYSIS AND DESIGN OF ALGORITHMS LAB

Assignment-1: Algorithm Foundations

Name: Rahul

Roll No.: 2401010257

Section: C

Specialisation: B.Tech CSE

Task 1: Algorithm Growth Observation

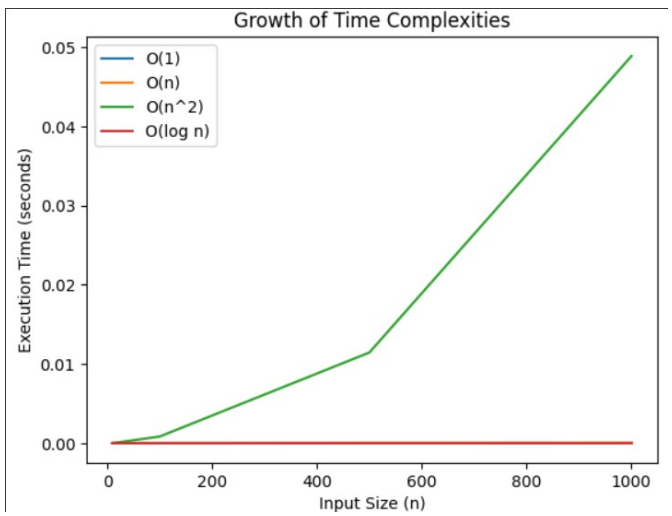
Implement O(1), O(n), O(n^2), O(log n) and measure execution time.

1. Constant Time : *O(1)*
2. Linear Time : *O(n)*
3. Quadratic Time : $O(n^2)$
4. Logarithmic Time : *O(log n)*

```
 import time import random
import matplotlib.pyplot as plt
import math  def
constant_time(n):
return n * 5  def
linear_time(n):
total = 0 for i in
range(n):
total += i
return total
 def
quadratic_time(n):
total = 0 for i in
range(n): for j in
range(n):
total += 1 return total
def logarithmic_time(n):
count = 0 while
n > 1: n //= 2
count += 1
return count
sizes = [10,
100, 500, 1000]
results =
{"O(1)": [],
"O(n)": [],
```

```
"O(n^2)": [],
"O(log n)": []}
 for n in sizes: # O(1) start = time.time()
constant_time(n)
results["O(1)"].append(time.time() - start)
 # O(n) start = time.time() linear_time(n)
results["O(n)"].append(time.time() - start)
 # O(n^2) start = time.time()
quadratic_time(n)
results["O(n^2)"].append(time.time() - start)
 # O(log n) start = time.time()
logarithmic_time(n) results["O(log
n)"].append(time.time() - start)
 for key in
results:
plt.plot(sizes, results[key], label=key)
 plt.xlabel("Input Size (n)")
plt.ylabel("Execution Time (seconds)")
plt.title("Growth of Time Complexities")
plt.legend() plt.show()
```

Output:



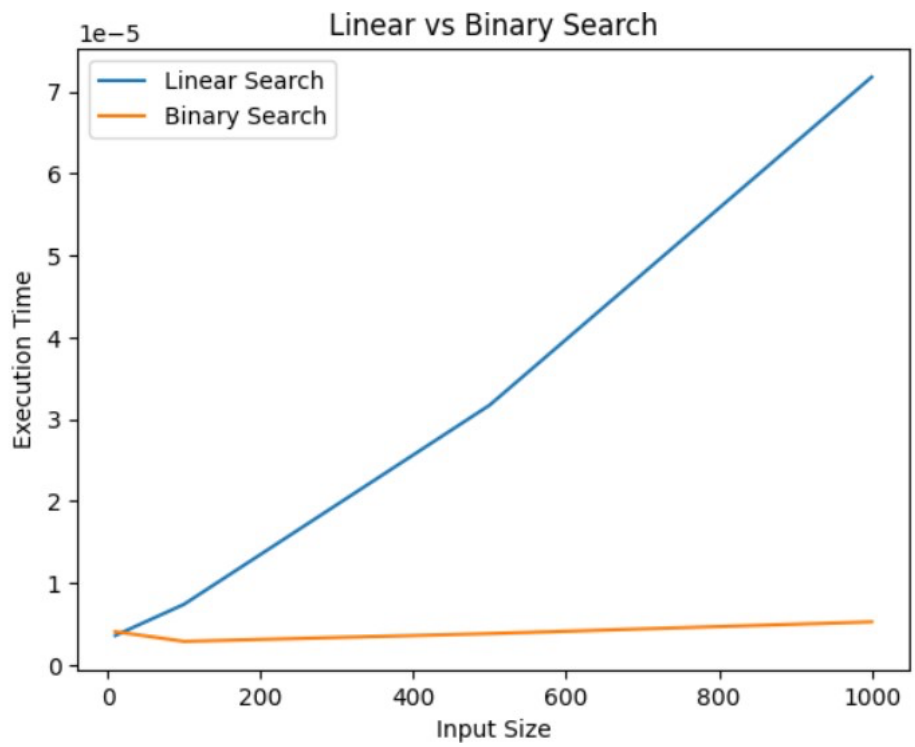
Task 2: Best, Average, and Worst Case Analysis

Implement Linear Search and Binary Search and compare execution times.

```
 def linear_search(arr, key):
for i in range(len(arr)):
if arr[i] == key:
return i     return -1  def
binary_search(arr, key):
low, high = 0, len(arr) - 1
while low <= high:
        mid = (low + high) // 2
if arr[mid] == key:
```

```
return mid         elif
arr[mid] < key:             low
= mid + 1         else:
high = mid - 1     return -1
sizes = [100, 500, 1000, 5000]
linear_times = [] binary_times
= []
 for n in
sizes:
    arr = sorted(random.sample(range(n*10), n))
start = time.time()     linear_search(arr, -1)
linear_times.append(time.time() - start)
     start = time.time()
binary_search(arr, -1)
binary_times.append(time.time() - start)
 plt.plot(sizes, linear_times, label="Linear
Search") plt.plot(sizes, binary_times, label="Binary
Search") plt.xlabel("Input Size")
plt.ylabel("Execution Time")
plt.legend() plt.title("Linear vs
Binary Search") plt.show()
```

Output:



Task 3: Recursion and Recurrence Validation

Implement Factorial, Fibonacci (Recursive), Fibonacci (DP).

```
 call_count =
0
 def factorial(n):     global
call_count     call_count +=
1     if n == 0:
return 1     return n *
factorial(n-1)
 def
fibonacci_recursive(n):
    global call_count
    call_count += 1
    if n <= 1:
        return n
return
fibonacci_recursive(n
-1) +
fibonacci_recursive(n
-2)
 def fibonacci_dp(n):
fib = [0, 1]     for i in
range(2, n+1):
        fib.append(fib[i-1] + fib[i-2])
return fib[n]
```

Output:

n	Recursive Calls (Fib)	Time (Recursive)	Time (DP)
10	High	High	Very Low
20	Very High	Very High	Low

Task 4: Solving Recurrences through Code

Implement recursive functions for given recurrences.

1. $T(n) = T(n/2) + n$
2. $T(n) = 2T(n/2) + n$

```
 call_count =
0
 def recurrence1(n):     global
call_count     call_count += 1
if n <= 1:         return 1
return recurrence1(n//2) + n
 def recurrence2(n):     global call_count
call_count += 1     if n <= 1:         return 1

    return recurrence2(n//2) + recurrence2(n//2) + n
```

Github: `https://github.com/rkadyan450/lab-assignment-ada-1x`